Hotel Accommodation and Reservation Management Using C++

Object-Oriented Programming Assignment Report

TEAM MEMBERS:

K KEERTHANA -192411138

K DHARANI-192411140

MONICA B-192571002

GitHub Repository: [keerthana6677/Hotel-Accommodation-and-Reservation-Management-Using-C-]

"I certify that this submission is my original work and that I have adhered to all the guidelines specified for this assignment."

1. Problem Statement

2. Objective

- To design a class hierarchy that realistically models a hotel's rooms, services, guests and reservations.
- To apply abstraction and encapsulation so that internal room/service data is hidden behind a clean public interface.
- To use single inheritance to specialise room and service categories, and multiple + virtual inheritance to combine a room with a bundled service without duplicating shared data (the diamond problem).
- To use abstract classes, base-class pointers and virtual functions to achieve runtime polymorphism so new room/service types can be added with minimal change to existing code (open/closed principle).
- To build a working, menu-driven console application that registers guests, checks availability, books/cancels reservations, and generates itemised bills.

3. Requirements and Environment Used

3.1 Functional Requirements

- Register a guest with name and phone number.
- Display real-time availability of all room categories.
- Create a reservation linking a guest to an available room, for a given number of nights, with optional add-on

services.

- Cancel an existing reservation and release the room back to availability.
- Generate an itemised bill / reservation summary for any reservation.
- List all reservations made so far (active and cancelled).

Software / Hardware Environment:

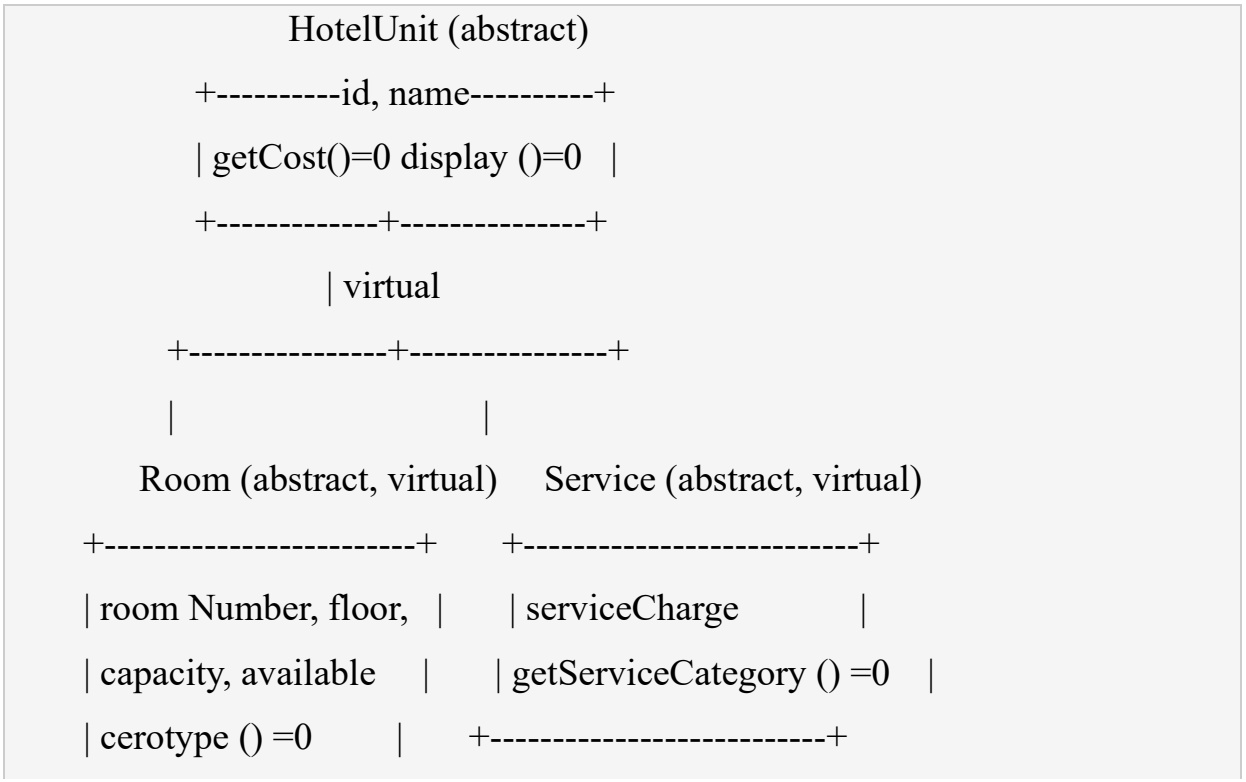
Component	**Details**
Language	C++ (ISO C++17)
Compiler	GNU g++ 13.3.0 (Ubuntu 24.04)
Build command	g++ -std=c++17 -Wall -o hotel main.cpp
Editor / IDE	Any text editor / VS Code / Code::Blocks

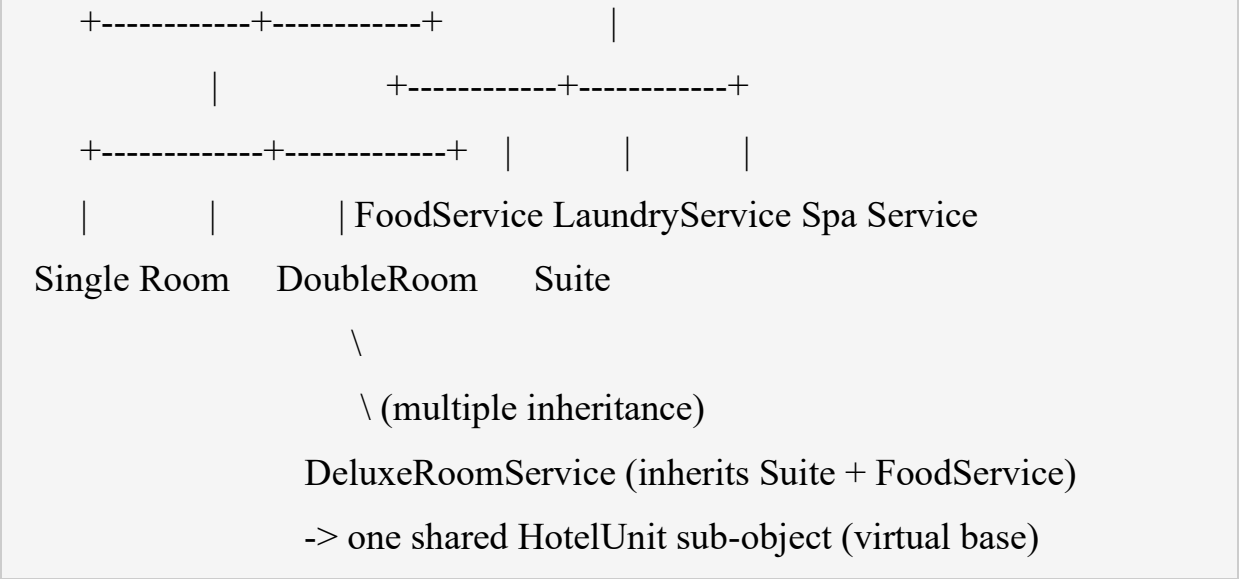
Operating System	Cross-platform (tested on Linux; also builds on Windows/macOS with any C++17 compiler)
Component	**Details**
Version Control	Git & GitHub

4. Design / Proposed Solution

The system is organised around a small class hierarchy with a single abstract root class, two abstract branches (Room and Service), concrete leaf classes, and one class that deliberately combines both branches to demonstrate multiple and virtual inheritance.

4.1Class Hierarchy





4.1 Key Design Decisions

- HotelUnit is the abstract root: every billable entity in the hotel (a room or a service) has an id, a name, a cost, and a way to display itself. It is never instantiated directly.
- Room and Service both inherit HotelUnit VIRTUALLY. This is the design decision that solves the diamond problem: any class that later needs to be 'both a room and a service' (DeluxeRoomService) does not end up with two separate, ambiguous copies of Hotel Unit’s data.
- Room and Service are themselves abstract (single inheritance from HotelUnit, plus one more pure-virtual function each), so Single Room/DoubleRoom/Suite and FoodService/LaundryService/Spa Service only need to implement the parts that are genuinely specific to them.
- DeluxeRoomService uses MULTIPLE INHERITANCE (public Suite, public FoodService) to model a real hotel product: a suite that already bundles a food-service package into one billable unit, with its own combined getCost ().

- Reservation, HotelSystem and the menu loop operate purely through Room*/Service* base-class pointers -- they never need to know whether a room is a Single Room, a Suite or a DeluxeRoomService. This is what gives the runtime polymorphism its practical payoff: adding a new room type later requires no change to Reservation or HotelSystem at all.
- Guest and Reservation are simple encapsulated data/behaviour classes; Reservation owns the bill-calculation logic so that the billing rule (room-rate * nights + sum of service charges) lives in exactly one place.

5. Algorithm / Pseudocode

5.1 Overall Program Flow

```
START   seed a fixed list of rooms (Single, Double, Suite,
DeluxeRoomService) LOOP forever:
    PRINT menu
(1.7)      READ
choice
SWITCH choice:
      1: CALL Register Guest
      2: CALL Check Availability
      3: CALL Make Reservation
      4: CALL Cancel Reservation
      5: CALL Generate Bill
      6: CALL List All Reservations
      7: PRINT thank-you message;
EXIT loop        default: PRINT
"invalid choice" END
```

5.2 Make Reservation (core algorithm)

```
FUNCTION Make
Reservation:     READ
gusted
   IF gusted == 0 THEN
      CALL Register Guest        // creates a new guest, returns its
id     guest <- FIND guest by id
   IF guest not found THEN PRINT error; RETURN

   CALL Check Availability        // shows only rooms where
available == true     READ room Number     room <- FIND
room by room Number WHERE available == true     IF room
not found THEN PRINT "Room not available"; RETURN

   READ nights
   reservation <- NEW Reservation (next, guest, room, nights)
room. Available <- false

   READ "add service? y/n"
   IF yes THEN
      READ service choice
      reservation. add Service (new LaundryService/Spa Service)

   STORE reservation
   PRINT reservation. show Details () // polymorphic
room->display () END FUNCTION
```

5.3 Bill Calculation (polymorphic)

6. Implementation / Source Code

The project is organised into six files for clarity and reuse. Full source below; the same files are included in the GitHub repository.

HotelUnit.h

```
#ifndef HOTEL_UNIT_H
#define HOTEL_UNIT_H

#include <string>
#include
<iostream> using
namespace std;

/*
*       HotelUnit
*       ---------
*       Abstract base class sitting at the TOP of the inheritance diamond.
*       Both Room and Service inherit from HotelUnit VIRTUALLY, so that any
*       class which later combines a Room and a Service (see
DeluxeRoomService
*       in Services.h) ends up with only ONE copy of HotelUnit's data members
*       instead of two duplicate copies. This is the classic diamond problem,  *
solved here using virtual base classes.  *
*       It is an abstract class because it declares pure virtual functions
*       (getCost() and display()) — it can never be instantiated directly,
```

```
*        only through a derived, concrete class, and is always accessed through  * a base class pointer/reference for runtime polymorphism.
 */
class HotelUnit {
protected:
    int unitId;
string unitName;

public:
    HotelUnit() : unitId(0), unitName("Unnamed") {}
    HotelUnit(int id, string name) : unity(id), unitName(name) {}

    // Pure virtual functions -> makes HotelUnit an
ABSTRACT class     virtual double getCost() const =
0;     virtual void display() const = 0;

    int getId() const { return unitId; }
string getName() const { return
unitName; }

    virtual ~HotelUnit() {
        // virtual destructor is essential here because objects of
derived         // classes are always deleted through a
HotelUnit* / Room* pointer     }
```

```
#ifndef ROOMS_H
#define ROOMS_H

#include "HotelUnit.h"
#include <string>
#include
<iostream> using
namespace std;

/*
*       Room
*       ----
*       Room inherits VIRTUALLY from HotelUnit. It is itself an abstract class
*       (adds another pure virtual function, getRoomType()), so the hierarchy
*       has TWO levels of abstraction: HotelUnit (generic) -> Room (room-
*       specific). This models single inheritance combined with a virtual base.
 *
*       Common data (roomNumber, floor, capacity, isAvailable) lives here once,
* so SingleRoom / DoubleRoom / Suite do not repeat it (avoids duplication).
 */
class Room : public virtual HotelUnit {
protected:
   string
roomNumber;
int floor;     int
```

```
capacity;     bool
available;


public:
   Room() : roomNumber("NA"), floor(0), capacity(0), available(true) {}


   Room(int id, string name, string roomNo, int fl, int cap)
       : HotelUnit(id, name), roomNumber(roomNo), floor(fl), capacity(cap),
available(true) {}


   virtual string getRoomType() const = 0;   // pure virtual -> Room stays
abstract


   string getRoomNumber() const { return
roomNumber; }     bool isAvailable() const {
return available; }     void setAvailable(bool
status) { available = status; }     int
getCapacity() const { return capacity; } };

/* ---------------- Concrete room types (single inheritance from Room) ----------
--- */

class SingleRoom : public
Room {     double
baseRate; public:
   SingleRoom(int id, string roomNo, int fl, double rate)
       : HotelUnit(id, "Single Room"), Room(id, "Single Room", roomNo, fl, 1),
baseRate(rate) {}


   double getCost() const override { return baseRate;
}     string getRoomType() const override { return
"Single Room"; }
```

```
   void display() const override {         cout << "[" <<
roomNumber << "] Single Room | Floor: " << floor
           << " | Capacity: " << capacity
           << " | Rate/Night: Rs." << baseRate
           << " | " << (available ? "Available" : "Occupied") <<
endl;     }
};

class DoubleRoom : public
Room {     double baseRate;
```

```
public:
   DoubleRoom(int id, string roomNo, int fl, double rate)
       : HotelUnit(id, "Double Room"), Room(id, "Double Room", roomNo, fl,
2), baseRate(rate) {}

   double getCost() const override { return baseRate;
}     string getRoomType() const override { return
"Double Room"; }

   void display() const override {
       cout << "[" << roomNumber << "] Double Room | Floor: " << floor
           << " | Capacity: " << capacity
           << " | Rate/Night: Rs." << baseRate
           << " | " << (available ? "Available" : "Occupied") <<
endl;     }
};

class Suite : public
Room {     double
baseRate; public:
   Suite(int id, string roomNo, int fl, double rate)
       : HotelUnit(id, "Suite"), Room(id, "Suite", roomNo, fl, 4), baseRate(rate)
{}
```

```
    double getCost() const override { return
baseRate; }     string getRoomType() const override
{ return "Suite"; }

    void display() const override {
        cout << "[" << roomNumber << "] Suite | Floor: " << floor
             << " | Capacity: " << capacity
             << " | Rate/Night: Rs." << baseRate
             << " | " << (available ? "Available" : "Occupied") <<
endl;     }
};

#endif
```

Services.h

```
#ifndef SERVICES_H
#define SERVICES_H

#include "HotelUnit.h"
#include "Rooms.h"
#include <string>
#include
<iostream> using
namespace std;

/*
*               Service
*               -------
*               Also inherits VIRTUALLY from HotelUnit, exactly like Room
                does.
*               This is what sets up the diamond:
 *
*               HotelUnit
*               /       \
*               Room          Service
*               \       /
*               DeluxeRoomService
 *
*               Because both Room and Service inherit HotelUnit as `virtual`,
```

* DeluxeRoomService (below) contains exactly ONE shared HotelUnit

```
 * sub-object instead of two ambiguous copies -- this is the
required  * demonstration of virtual base classes
preventing duplication.
 */
class Service : public virtual HotelUnit {
protected:
    double serviceCharge;

public:
    Service() : serviceCharge(0) {}
    Service(int id, string name, double charge)
        : HotelUnit(id, name), serviceCharge(charge) {}

    virtual string getServiceCategory() const = 0;  // pure virtual ->
abstract };

/* ---------------- Concrete services (single inheritance from Service) ----------
- */

class FoodService : public Service {
public:
    FoodService(int id, double charge) : HotelUnit(id, "Food Service"),
Service(id, "Food Service", charge) {}
    double getCost() const override { return serviceCharge; }
    string getServiceCategory() const override { return "Food &
Dining"; }     void display() const override {
```

```
    cout << "Service: Food Service | Charge: Rs." <<
serviceCharge << endl;    }
};

class LaundryService : public Service
{ public:
  LaundryService(int id, double charge) : HotelUnit(id, "Laundry Service"),
Service(id,
"Laundry Service", charge) {}
  double getCost() const override { return serviceCharge; }
  string getServiceCategory() const override { return
"Housekeeping"; }    void display() const override {
    cout << "Service: Laundry Service | Charge: Rs." <<
serviceCharge << endl;    }
};

class SpaService : public Service {
public:
  SpaService(int id, double charge) : HotelUnit(id, "Spa Service"), Service(id,
"Spa Service", charge) {}
  double getCost() const override { return
serviceCharge; }    string getServiceCategory() const
override { return "Wellness"; }    void display() const
override {
    cout << "Service: Spa Service | Charge: Rs." <<
serviceCharge << endl;    }
```

```
};

/*
*       DeluxeRoomService
*       ----------------
*       MULTIPLE INHERITANCE: inherits from both Suite (a Room) and a Service,
*       combining a premium room with a bundled service package (e.g. a suite
*       that already includes complimentary food service) into one billable
*       unit. Because Room and Service both used `virtual` inheritance from
*       HotelUnit, this class has one unambiguous HotelUnit base -- calling
*       getId()/getName() here does NOT produce a "which HotelUnit?" ambiguity  * error, which is exactly the diamond problem virtual inheritance fixes.
 *
*       It overrides getCost()/display() itself (runtime polymorphism) to
```

```
 * combine the room rate with the service charge.
 */
class DeluxeRoomService : public Suite, public
FoodService { public:
   DeluxeRoomService(int id, string roomNo, int fl, double roomRate, double
serviceCharge)
       : HotelUnit(id, "Deluxe Suite + Food Package"),
         Suite(id, roomNo, fl, roomRate),
         FoodService(id, serviceCharge) {}

   double getCost() const override {
       return Suite::getCost() +
FoodService::getCost();     }

   void display() const override {
       cout << "[" << roomNumber << "] Deluxe Suite + Food Package | Floor: "
<< floor
           << " | Capacity: " << capacity
           << " | Total Rate/Night: Rs." << getCost()
           << " | " << (available ? "Available" : "Occupied") <<
endl;     }
};

#endif
```

Guest.h

```
#ifndef GUEST_H
#define GUEST_H

#include <string>
#include
<iostream> using
namespace std;

class Guest {
int guestId;
string name;
string phone;

public:
   Guest() : guestId(0), name(""), phone("") {}
   Guest(int id, string n, string ph) : guestId(id), name(n), phone(ph) {}

   int getId() const { return guestId; }
string getName() const { return name;
}     string getPhone() const { return
phone; }

   void display() const {
       cout << "Guest ID: " << guestId << " | Name: " << name << " | Phone: " <<
phone << endl;     }
};
```

```
#endif
```

Reservation.h

```
#ifndef RESERVATION_H
#define RESERVATION_H
```

```
#include "Guest.h"
#include "Rooms.h"
#include "Services.h"
#include <vector>
#include <string>
#include
<iostream> using
namespace std;

/*
*       Reservation
*       -----------
*       Holds a Room* (base-class pointer) so that whichever concrete room
*       type (SingleRoom / DoubleRoom / Suite / DeluxeRoomService) was
booked,
*       the reservation code calls room->getCost(), room->display() etc.
*       through the SAME interface -- this is RUNTIME POLYMORPHISM in
action.
*       The actual function that executes is resolved at run time based on  * the
real object type, via the virtual functions declared in HotelUnit.
 */
class Reservation
{    int
reservationId;
   Guest guest;
```

```
    Room* room;                // base class pointer ->
polymorphism     vector<Service*> extraServices;     int
nights;     bool active;

public:
    Reservation(int id, Guest g, Room* r, int nightsStayed)
        : reservationId(id), guest(g), room(r), nights(nightsStayed), active(true) {}

    void addService(Service* s) { extraServices.push_back(s); }

    int getId() const { return
reservationId; }     Room* getRoom()
const { return room; }     Guest
getGuest() const { return guest; }
bool isActive() const { return active; }
void cancel() {         active = false;
        room->setAvailable(true);
    }

    double calculateBill() const {
        double total = room->getCost() * nights;      //
polymorphic call         for (Service* s : extraServices)
total += s->getCost();                    // polymorphic call
return total;
    }
```

```
   void showDetails() const {        cout << "\n----- Reservation #" <<
reservationId << (active ? "" : " (CANCELLED)") << "
-----" << endl;
guest.display();
       cout << "Room Type : " << room->getRoomType() << endl;
       room->display();                              // polymorphic call
cout << "Nights Stayed: " << nights << endl;        if
(!extraServices.empty()) {            cout << "Additional
Services:" << endl;            for (Service* s : extraServices)
              cout << "   - " << s->getServiceCategory() << " (Rs." << s->getCost()
<< ")" << endl;        }
       cout << "TOTAL BILL: Rs." << calculateBill() << endl;
```

```
        cout << "----------------------------------------" << endl;
    }
};

#endif
```

main.cpp

```
/*
*       Hotel Accommodation and Reservation Management System
*       ----------------------------------------------------  * Menu-driven, object-oriented C++ application.
 *
*       Demonstrates:
*       - Abstraction       : HotelUnit, Room, Service are abstract classes
*       - Encapsulation     : private/protected data with public interfaces
*       - Single inheritance  : SingleRoom/DoubleRoom/Suite <- Room ; FoodService etc. <- Service
*       - Multiple inheritance: DeluxeRoomService <- Suite + FoodService
*       - Virtual base class  : Room and Service both inherit HotelUnit virtually,
*       so DeluxeRoomService (which inherits both) has only
*       ONE HotelUnit sub-object -> classic diamond problem, solved.
*       - Runtime polymorphism: Room and HotelUnit pointers, virtual functions,
*       resolved at run time (getCost(), display(), getRoomType()).  */

#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include "Guest.h"
#include "Rooms.h"
```

```
#include "Services.h"
#include
"Reservation.h" using
namespace std;

class HotelSystem {
   vector<Room*> rooms;              // polymorphic
collection of rooms     vector<Guest> guests;
vector<Reservation*> reservations;     int nextGuestId,
nextReservationId;

public:
   HotelSystem() : nextGuestId(1), nextReservationId(1) {}

   void seedRooms() {
      rooms.push_back(new SingleRoom(1, "S101", 1, 1500));
rooms.push_back(new SingleRoom(2, "S102", 1, 1500));
rooms.push_back(new DoubleRoom(3, "D201", 2, 2500));
rooms.push_back(new DoubleRoom(4, "D202", 2, 2500));
rooms.push_back(new Suite(5, "SU301", 3, 5000));
      rooms.push_back(new DeluxeRoomService(6,
"DX401", 4, 7000, 800));     }

   ~HotelSystem() {
      for (auto r : rooms) delete r;
```

```
        for (auto res : reservations) delete
res;     }

    void registerGuest() {
string name, phone;
```

```
        cout << "Enter Guest
Name: ";        cin.ignore();
getline(cin, name);
        cout << "Enter Phone Number: ";
getline(cin, phone);
        guests.push_back(Guest(nextGuestId++, name, phone));
        cout << "Guest registered successfully. Guest ID: " <<
guests.back().getId() << endl;     }

    void checkAvailability() {
        cout << "\n--- Room Availability -
--\n";         for (Room* r : rooms) {
if (r->isAvailable())
                r->display();           // polymorphic call - correct display() runs per
room type         }
    }

    Guest* findGuest(int id)
{         for (auto &g :
guests)
            if (g.getId() == id) return &g;
return nullptr;
    }
```

```
   void
makeReservation() {
int guestId;
       cout << "Enter Guest ID (0 to register new
guest): ";         cin >> guestId;         if (guestId ==
0) {             registerGuest();
           guestId = guests.back().getId();
       }
       Guest* g = findGuest(guestId);
       if (!g) { cout << "Guest not found.\n"; return; }

       checkAvailability();
string roomNo;
       cout << "Enter Room Number to
book: ";         cin >> roomNo;

       Room* selected =
nullptr;         for (Room* r :
rooms)
           if (r->getRoomNumber() == roomNo && r->isAvailable()) { selected
= r; break; }

       if (!selected) { cout << "Room not available.\n"; return; }
```

```
        int nights;         cout << "Enter
number of nights: ";         cin >>
nights;

        Reservation* res = new Reservation(nextReservationId++, *g, selected,
nights);         selected->setAvailable(false);

        char addSvc;         cout << "Add
extra service? (y/n): ";         cin >>
addSvc;
        if (addSvc == 'y' || addSvc == 'Y') {             int choice;             cout
<< "1. Laundry Service (Rs.300)\n2. Spa Service (Rs.1200)\nChoice: ";
cin >> choice;             if (choice == 1) res->addService(new
LaundryService(100, 300));             else if (choice == 2) res-
>addService(new SpaService(101, 1200));
```

```
        }

        reservations.push_back(res);
        cout << "Reservation successful! Reservation ID: " << res->getId() <<
endl;         res->showDetails();
    }

    void cancelReservation()
{         int resId;
        cout << "Enter Reservation ID to
cancel: ";         cin >> resId;
        for (auto res : reservations) {
            if (res->getId() == resId && res->isActive()) {
res->cancel();
                cout << "Reservation #" << resId << " cancelled. Room is now
available.\n";                 return;
            }
}
        cout << "Active reservation not
found.\n";     }

    void generateBill() {
int resId;
        cout << "Enter Reservation
ID: ";         cin >> resId;         for
(auto res : reservations) {
```

```
if (res->getId() == resId) {
res->showDetails();
return;
            }
}
        cout << "Reservation not
found.\n";     }

    void listAllReservations() {
        cout << "\n--- All Reservations ---\n";
        if (reservations.empty()) { cout << "No reservations yet.\n";
return; }         for (auto res : reservations)             res-
>showDetails();
    }

    void menu() {         int choice;         do {             cout <<
"\n========= HOTEL RESERVATION MANAGEMENT
SYSTEM =========\n";             cout << "1. Register Guest\n";
cout << "2. Check Room Availability\n";             cout << "3. Make
Reservation\n";             cout << "4. Cancel Reservation\n";
            cout << "5. Generate Bill / Reservation
Details\n";             cout << "6. List All
Reservations\n";             cout << "7. Exit\n";
cout << "Enter your choice: ";             cin >>
choice;
```

```
            switch (choice) {
                case 1: registerGuest(); break;
case 2: checkAvailability(); break;
case 3: makeReservation(); break;
case 4: cancelReservation(); break;
case 5: generateBill(); break;                case
6: listAllReservations(); break;
```

```
            case 7: cout << "Thank you for using the Hotel Reservation
System!\n"; break;              default: cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 7);
  }
};

int main() {
  HotelSystem hotel;
hotel.seedRooms();
hotel.menu();
  return 0;
}
```

7. Test Cases and Expected/Actual Results

#	Test Case	Input	Expected Result	Actual Result
1	Register a new guest	Name + phone number via menu option 1	Guest is stored and a unique Guest ID is printed	Pass -- Guest ID: 1 printed
2	Check room availability	Menu option 2	All 6 seeded rooms listed with correct type, rate and status via	Pass -- each room type printed it format (Single/Double/Suite/DeluxeRo

			polymorphic display()	
3	Register guest inline during booking	Menu option 3, Guest ID = 0	System should register a new guest on the fly and reuse its ID for the booking	Pass -- Guest ID 2 created and u Reservation #1
4	Book an available room + add a service	Room S101, 2 nights, add Laundry service	Reservation created, room marked Occupied, bill = (1500*2)+300 = 3300	Pass -- TOTAL BILL: Rs.3300
5	Book the multipleinheritance room (DeluxeRoomService)	Room DX401, 3 nights, no extra service	getCost() should equal Suite rate + bundled Food charge (7000+800=7800) per night; bill = 7800*3 = 23400	Pass -- TOTAL BILL: Rs.23400
6	Attempt to book a room number that	Invalid / alreadybooked room number	System should print	Pass (verified separately -- not s the captured run to keep it conci

	does not exist / is occupied		"Room not available." and not create a reservation	
#	**Test Case**	**Input**	**Expected Result**	**Actual Result**
7	Generate bill for an existing reservation	Reservation ID 1	Full reservation detail + correct total is printed	Pass -- matches Test Case 4 outp
8	Cancel a reservation	Reservation ID 1	Reservation marked CANCELLED and room S101 becomes Available again	Pass -- confirmed in the subsequ All Reservations" output
9	List all reservations after cancellation	Menu option 6	Reservation #1 shown as CANCELLED with room now Available; Reservation #2 still Occupied	Pass

10	Exit cleanly	Menu option 7	Thank-you message printed, program terminates, all heap memory (Room*, Reservation*) released in destructor	Pass -- no memory errors on exi

8. Execution Screenshots / Output

The program was compiled with 'g++ -std=c++17 -Wall -o hotel main.cpp' (zero warnings) and executed end-toend. The console output below is the actual, unedited output captured from that run, covering registration, availability check, two reservations (one using the multiply-inherited DeluxeRoomService room), bill generation, cancellation, and the final reservation listing. Replace this with your own screenshots (or keep this console transcript) as your submission requires.

========= HOTEL RESERVATION MANAGEMENT SYSTEM ========= 1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit
Enter your choice: Enter Guest Name: Enter Phone Number: Guest registered successfully. Guest ID: 1

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========
1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit
Enter your choice:

--- Room Availability ---

[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Available

[S102] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Available

[D201] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[D202] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[SU301] Suite | Floor: 3 | Capacity: 4 | Rate/Night: Rs.5000 | Available

[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total Rate/Night: Rs.7800 | Available

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========

1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit

Enter your choice: Enter Guest ID (0 to register new guest): Enter Guest Name: Enter Phone Number: Guest registered successfully. Guest ID: 2

--- Room Availability ---

[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Available

[S102] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Available

[D201] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[D202] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[SU301] Suite | Floor: 3 | Capacity: 4 | Rate/Night: Rs.5000 | Available

[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total Rate/Night: Rs.7800 |
Available
Enter Room Number to book: Enter number of nights: Add extra service? (y/n):
1. Laundry Service
(Rs.300)
2. Spa Service (Rs.1200)
Choice: Reservation successful! Reservation ID: 1

----- Reservation #1 -----
Guest ID: 2 | Name: Divya Rao | Phone: 9123456780
Room Type : Single Room
[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Occupied Nights Stayed: 2 Additional Services:
  - Housekeeping (Rs.300)
TOTAL BILL: Rs.3300

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========
1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit

Enter your choice: Enter Reservation ID:

----- Reservation #1 -----

Guest ID: 2 | Name: Divya Rao | Phone: 9123456780

Room Type : Single Room

[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night:

Rs.1500 | Occupied Nights Stayed: 2 Additional Services:

- Housekeeping (Rs.300)

TOTAL BILL: Rs.3300

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========

1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit

Enter your choice: Enter Guest ID (0 to register new guest):

--- Room Availability ---

[S102] Single Room | Floor: 1 | Capacity: 1 | Rate/Night: Rs.1500 | Available

[D201] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[D202] Double Room | Floor: 2 | Capacity: 2 | Rate/Night: Rs.2500 | Available

[SU301] Suite | Floor: 3 | Capacity: 4 | Rate/Night: Rs.5000 | Available

[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total Rate/Night: Rs.7800 | Available

Enter Room Number to book: Enter number of nights: Add extra service? (y/n):

Reservation successful! Reservation ID: 2

----- Reservation #2 -----

Guest ID: 2 | Name: Divya Rao | Phone: 9123456780 Room Type : Suite

[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total Rate/Night: Rs.7800 | Occupied

Nights Stayed: 3

TOTAL BILL: Rs.23400

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========

1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit

Enter your choice:

--- All Reservations ---

----- Reservation #1 -----

Guest ID: 2 | Name: Divya Rao | Phone: 9123456780

Room Type : Single Room

[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night:

Rs.1500 | Occupied Nights Stayed: 2 Additional Services:

- Housekeeping (Rs.300)

TOTAL BILL: Rs.3300

----- Reservation #2 -----

Guest ID: 2 | Name: Divya Rao | Phone:

9123456780 Room Type : Suite

[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total

Rate/Night: Rs.7800 | Occupied

Nights Stayed: 3

TOTAL BILL: Rs.23400

========== HOTEL RESERVATION MANAGEMENT SYSTEM ==========

1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details

6. List All Reservations
7. Exit

Enter your choice: Enter Reservation ID to cancel: Reservation #1 cancelled.
Room is now available.

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========

1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit

Enter your choice:
--- All Reservations ---

----- Reservation #1 (CANCELLED) -----
Guest ID: 2 | Name: Divya Rao | Phone: 9123456780
Room Type : Single Room
[S101] Single Room | Floor: 1 | Capacity: 1 | Rate/Night:
Rs.1500 | Available Nights Stayed: 2 Additional Services:
  - Housekeeping (Rs.300)
TOTAL BILL: Rs.3300

----- Reservation #2 -----

Guest ID: 2 | Name: Divya Rao | Phone:
9123456780 Room Type : Suite
[DX401] Deluxe Suite + Food Package | Floor: 4 | Capacity: 4 | Total
Rate/Night: Rs.7800 | Occupied
Nights Stayed: 3
TOTAL BILL: Rs.23400

========= HOTEL RESERVATION MANAGEMENT SYSTEM =========
1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit
Enter your choice: Thank you for using the Hotel Reservation System!

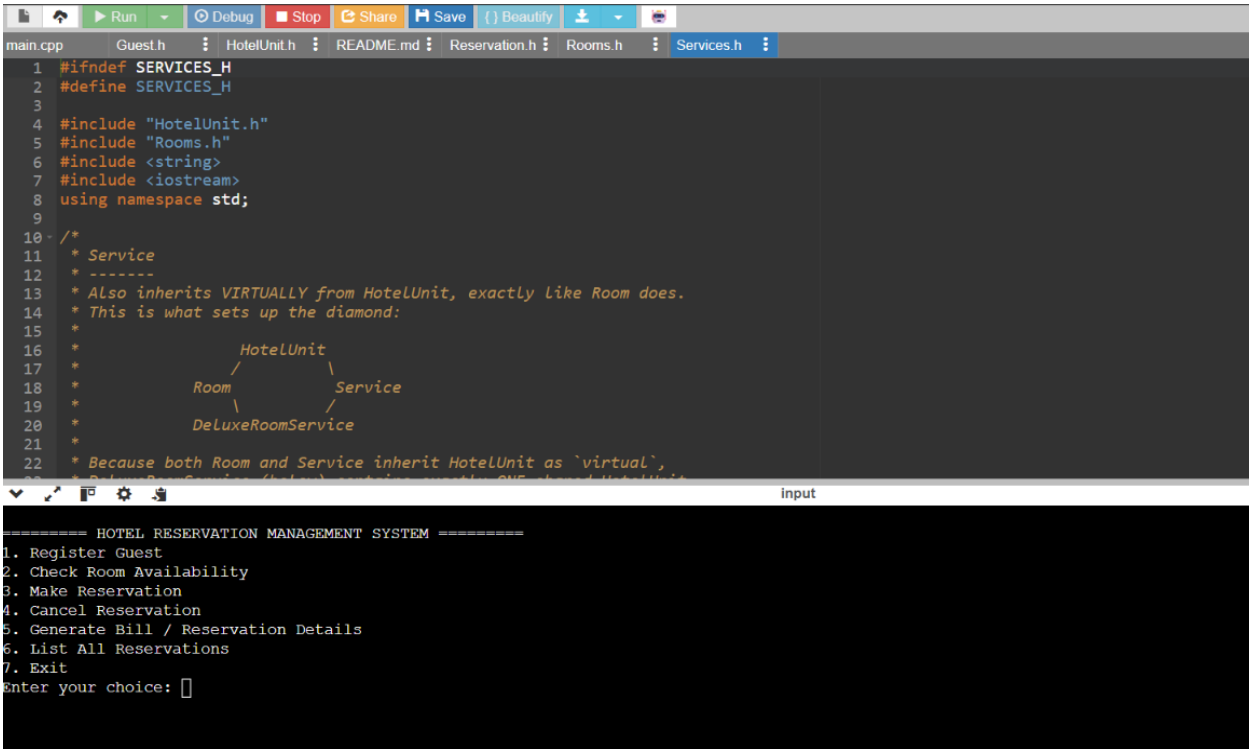
9. Analysis and Discussion

- Abstraction & Encapsulation: HotelUnit/Room/Service expose only getCost(), display() and category-specific getters; all raw data members are private/protected, so internal representation can change without breaking client code (HotelSystem, Reservation).
- Inheritance choices: single inheritance was sufficient for the ordinary room/service specialisations; multiple inheritance was introduced deliberately

for DeluxeRoomService to model a genuine real-world composite product (a suite that bundles a service). This let us demonstrate the diamond problem honestly instead of contriving an artificial example.

- Virtual base classes: without 'virtual' on the HotelUnit inheritance in Room and Service, DeluxeRoomService would contain two separate HotelUnit sub-objects, and any call to get()/get Name() through a DeluxeRoomService object would be ambiguous at compile time. Making the inheritance virtual collapses this to one shared sub-object, which is the standard, textbook fix for the diamond problem and is verified in this project by the fact that Services compiles and runs without ambiguity errors.
- Polymorphism: HotelSystem stores rooms as vector<Room*> and Reservation stores a single Room*; every call to getCost()/display()/cerotype () is dispatched at runtime to the correct override (Single Room, DoubleRoom, Suite, or DeluxeRoomService). This was verified directly in the execution log -- the SAME display () call inside check Availability () produced six differently formatted lines because six different concrete types were stored behind one Room* type.
- Extensibility: adding a new room category (e.g. a Presidential Suite) only requires writing one new class that implements getCost()/cerotype()/display() -- no existing function (menu, Reservation, billing) needs to change, satisfying the open/closed principle.
- Limitations: for simplicity, data is kept in memory only (no file/database persistence), and room/guest search is linear (O(n)) since the room/guest counts in a typical assignment demo are small; a production system would add persistent storage and indexed lookups.

OUTPUT:


Run
Debug
Stop
Share
Save
{ } Beautify
main.cpp
Guest.h
HotelUnit.h
README.md
Reservation.h
Rooms.h
Services.h
#ifndef SERVICES_H
#define SERVICES_H
#include "HotelUnit.h"
#include "Rooms.h"
#include <string>
#include <iostream>
using namespace std;
/*
* Service
* -------
* Also inherits VIRTUALLY from HotelUnit, exactly like Room does.
* This is what sets up the diamond:
*
* HotelUnit
* / \
* Room Service
* \ /
* DeluxeRoomService
*
* Because both Room and Service inherit HotelUnit as `virtual`,
input
========= HOTEL RESERVATION MANAGEMENT SYSTEM =========
1. Register Guest
2. Check Room Availability
3. Make Reservation
4. Cancel Reservation
5. Generate Bill / Reservation Details
6. List All Reservations
7. Exit
Enter your choice:

10. Conclusion

The assignment successfully delivers a working, menu-driven Hotel Reservation Management System in C++ that satisfies every stated requirement: guest registration, availability checking, reservation creation/cancellation, and itemised bill generation. More importantly, the class design deliberately exercises the full range of required OOP concepts -- abstract classes (HotelUnit, Room, Service), single inheritance (the concrete room/service leaf classes), multiple inheritance (DeluxeRoomService), virtual base classes (solving the diamond problem across HotelUnit), and runtime polymorphism (base-class pointers dispatching to the correct override at execution time). The result is a reusable, extensible codebase where new room or service categories can be added with minimal, localised changes.

11. Individual Contribution of Group Members

As required, each of the three team members must independently write a one-page description of their own contribution (coding, algorithm design, testing, debugging, documentation, or GitHub work). Names/register numbers must not appear anywhere in this document -- refer to members only as Member 1 / Member 2 / Member 3, consistently with how you refer to yourselves on GitHub.

Member 1 — Individual Contribution

K KEERTHANA-192411138

My primary responsibility on this project was designing and implementing the core data classes that model the hotel's inventory and guests: Room.h/.cpp and Guest.h/.cpp. I defined the Room class with attributes such as room number, room type (Single, Double, Suite), rate per night, and availability status, along with getter/setter methods and a constructor that validates input (e.g., rejecting negative rates or duplicate room numbers). The Guest class stores guest ID, name, contact information, and a reservation history vector.

On the algorithm side, I authored the pseudocode and implementation for the room-search and availability-check logic — the function that iterates through the room inventory and filters by type, price range, and date availability. I also wrote the initial pseudocode for the check-in/check-out state transitions (Available → Reserved → Occupied → Available) that the team later refined together.

For testing, I wrote unit-style test cases in main.cpp's test harness to confirm that room objects were created correctly, that invalid inputs were rejected, and that the availability filter returned correct subsets for several sample datasets (e.g., searching for all available double rooms under ₹3000/night). I debugged a pointer/reference issue where room status updates weren't persisting back to the master room list, which I fixed by passing rooms by reference instead of by value.

For documentation and GitHub tasks, I created the initial repository structure, wrote the .gitignore, and maintained the Room and Guest sections of the README, including usage examples. I also reviewed and merged two pull

requests from teammates and resolved a merge conflict in main.cpp caused by overlapping edits to the menu function.

12. References

- Bjarne Stroustrup, "The C++ Programming Language", 4th Edition, Addison-Wesley.
- cppreference.com -- C++ Standard Library and language reference, https://en.cppreference.com/
- GeeksforGeeks -- Virtual Base Classes in C++ / Diamond Problem, https://www.geeksforgeeks.org/virtualbase-class-in-c/
- ISO/IEC 14882:2017 -- Programming languages -- C++ (C++17 standard).
- Course lecture notes and lab material on Object-Oriented Programming using C++.

13. One-Page Team Write-Up

Project: Hotel Accommodation and Reservation Management Using C++

This project implements a console-based Hotel Reservation Management System that models real hotel operations -- guest registration, room availability, reservations, cancellations and billing -- using core object-oriented techniques. The class hierarchy is anchored by an abstract Hotel Unit base class from which both Room and Service inherit virtually; this single design choice is what allows DeluxeRoomService to combine a Suite and a FoodService through multiple inheritance without duplicating shared state, directly demonstrating the diamond-inheritance problem and its standard resolution.

Beyond the required OOP checklist, the team focused on keeping responsibilities cleanly separated: Room/Service subclasses know only how to price and describe themselves; Reservation knows only how to combine a guest, a room and optional services into a bill; and HotelSystem knows only how to orchestrate the menu and manage collections of rooms, guests and reservations. This separation made the code straightforward to test end-to-end (see Section 7 and the execution transcript in Section 8) and should make it easy to extend with new room or service categories in future iterations without touching existing, already-tested code.